Listing 10.1. An example test for concurrent `push()` and `pop()` calls on a queue

```
void test_concurrent_push_and_pop_on_empty_queue()
{
    threadsafe_queue<int> q;                                    <-1

    std::promise<void> go,push_ready,pop_ready;                 <-2
    std::shared_future<void> ready(go.get_future());            <-3

    std::future<void> push_done;                                <-4
    std::future<int> pop_done;

    try
    {
        push_done=std::async(std::launch::async,                <-5
                             [&q,ready,&push_ready]()
                             {
                                 push_ready.set_value();
                                 ready.wait();
                                 q.push(42);
                             }
            );
        pop_done=std::async(std::launch::async,                 <-6
                            [&q,ready,&pop_ready]()
                            {

                                pop_ready.set_value();
                                ready.wait();
                                return q.pop();                 <-7
                            }
            );
        push_ready.get_future().wait();                         <-8
        pop_ready.get_future().wait();
        go.set_value();                                         <-9

        push_done.get();                                        <-10
        assert(pop_done.get()==42);                             <-11
        assert(q.empty());
    }
    catch(...)
    {
        go.set_value();                                         <-12
        throw;
    }
}
```